Int J Syst Assur Eng Manag (May 2024) 15(5):1802–1814
https://doi.org/10.1007/s13198-023-02119-4

ORIGINAL ARTICLE

Machine learning based intrusion detection system for IoMT

Priyesh Kulshrestha[1] · **T. V. Vijay Kumar**[1]

Received: 18 May 2023 / Revised: 7 August 2023 / Accepted: 18 August 2023 / Published online: 7 September 2023

Abstract Millennials have the advantage of accessing readily available modern scientific advancements, particularly in technology. One of these technologies that encompasses varied functionalities is the Internet of Things (IoT). In the midst of the Covid-19 pandemic, IoT, specifically Internet of Medical Things (IoMT), had pivotal significance in monitoring and tracking different health parameters. It autonomously manages an individual's health data and stores the same as Electronic Health Records (EHRs). However, the networking protocols used by IoMT are not adequate enough to ensure the security and privacy of EHRs. Consequently, such technology is susceptible to cyber-attacks, which have become more prevalent over time and have taken various forms, that generally the stakeholders are not aware of. This paper introduces machine learning-driven intrusion detection systems as a solution to tackle this issue. The focus of this study is on devising a Machine Learning (ML) oriented Intrusion Detection System (IDS) designed to identify cyber-attacks targeting IoMT based systems. Several classification based ML techniques such as Multinomial Naive Bayes, Logistic Regression, Logistic Regression with Stochastic Gradient Descent, Linear Support Vector Classification, Decision Tree, Ensemble Voting Classifier, Bagging, Random Forest, Adaptive Boosting, Gradient Boosting and Extreme Gradient Boosting were used, whereupon the Adaptive Boosting was experimentally found to perform the best on performance metrics such as accuracy, precision, recall, F1-score, False Detection Rate (FDR) and False Positive Rate (FPR). Further, it was found that Adaptive boosting based IDS for IoMT performed comparatively better than the existing ToN_IoT based IDS models on performance metrics such as accuracy, F1-score, FPR and FDR.

Keywords IoT · IoMT · EHR · Cyber-security · Intrusion detection systems

1 Introduction

Technological intervention has helped humankind to achieve significant industrial outputs. The evolution from the era of industry 1.0 to industry 4.0 has improved lives drastically. Historically, the first industrial revolution involved steam-powered mechanical equipment while the second industrial revolution was blessed with electrical energy. The third industrial revolution enhanced the computing power of industries with the advent of information technology. The contemporary world is currently experiencing industrial revolution 4.0 that is based on the power of advanced technological support of Cloud Computing, Robotics, Internet of Things (IoT), Artificial Intelligence (AI), Big Data Analytics etc. (Sharma and Singh 2020). These technologies have embedded smartness and proficiency in several commercial streams such as Manufacturing, Finance, Professional Services, Energy, Retail, Healthcare, Transportation, Government, Education, Media, etc. These streams impact our lives directly or indirectly. Especially in the Healthcare industry which has a direct association with human wellbeing.

In recent times, the Covid-19 pandemic has brought out the importance of information technology (IT) especially its reach. On a positive side, this pandemic has been able to pin-point the loopholes in the existing technologies

✉ T. V. Vijay Kumar
tvvijaykumar@hotmail.com

Priyesh Kulshrestha
priyeshkul100@gmail.com

[1] School of Computer and Systems Sciences, Jawaharlal Nehru University, New Delhi 110067, India

and frameworks. Somehow, Healthcare 4.0 is managing to address such problems by providing intelligence to the systems. Large amounts of Electronic Health Records (EHRs) are analyzed and stored remotely with the help of cloud computing (Kioskli et al. 2021). The ecosystem of healthcare management in Healthcare 4.0, with the help of IoT technology, is referred to as IoMT. The invasion of monitoring devices, in our daily routines associated with the human body, bring to the table the factor of vulnerability to cyber threats and attacks.

According to IBM's security X force report (IBM Security 2022), the Healthcare industry is experiencing major cyber threats, with recorded stats of its threat ranking moving from 10th in 2019 to 6th in 2021 amongst all other industries such as Manufacturing, Finance, Professional Services, Energy, Retail, Transportation, Government, Education and Media. Healthcare cyber security issues mainly revolve around the security of EHRs primarily by stopping adversaries to penetrate into the IoMT environment or network. Stakeholders try to detect intrusions with network traffic analyzer software and hardware. Artificial intelligent systems can help in detecting infiltration with much greater proficiency. The objective of this research is to use ensemble machine learning techniques to model an intrusion detection mechanism.

A machine learning (ML) based IDS for IoMT is proposed in this paper. Rather than depending on one model, ensemble learning models create a multitude of base models for better predictions (Geron 2019). This machine learning model can be installed at the network traffic analyzer at different nodes in the architecture of IoMT (Zachos et al. 2021). The ensemble learning model is trained and evaluated using the ToN_IoT Network dataset (Moustafa 2021). The dataset undergoes preprocessing and is then subjected to the feature selection process. After selecting the features, the models would be trained and tested using fivefold cross-validations. This is followed by evaluating the proposed models on various performance metrics. The best performing model, thereafter, could be installed at the nodes in the architecture of IoMT for making timely and accurate attack predictions.

The structure of the paper is outlined as follows. Section 2 encompasses the literature review, which includes architecture of IoMT, overview of Intrusion Detections Systems (IDS), existing intrusion detection frameworks and models, brief discussion on machine learning and the ensemble learning methods used in this paper. Section 3 discusses the proposed ML based IDS for IoMT, which includes the ToN_IoT Network dataset description, data pre-processing, classification techniques and performance evaluation. Experimental results are discussed in Sect. 4. Section 5 is the conclusion.

2 Literature review

IoMT inherited cyber issues from its parent concept of IoT. The issues here become more severe considering the direct involvement of a person. Healthcare management systems use EHRs which are at stake of vulnerability in the present IoMT (Kandasamy et al. 2022). The heterogeneous environment of IoMT makes it insecure (Aldhaheri et al. 2020). For a better insight into making intelligent IDSs, the subsequent sections will provide sufficient knowledge of the architecture of IoMT blended with an overview of IDS, some previous solutions for intrusion detection and a supportive theory of ensemble learning methods which are used by the model proposed in this paper.

2.1 Architecture of IoMT

IoMT has a 3-layered architecture which comprises of the Device layer, the Fog layer and the Cloud layer (Kulshrestha et al. 2023), as depicted in Fig. 1. The Device layer has physical devices that collect and transmit medical data such as wearables and medical sensors. The Fog Layer acts as a gateway between the Device layer and the Cloud layer, which is responsible for receiving and transmitting data, performing data pre-processing, and handling security and data privacy. The Cloud Layer is the central repository for storing, analyzing medical data and providing access to authorized users such as healthcare providers and researchers (Lee et al. 2021).

The Fog Layer is generally identified as the major point of vulnerability. To address such vulnerabilities, an intelligent IDS is required. IDSs are classified on three different basis (Liu et al. 2018), as depicted in Fig. 2. On the basis of input data source, it is categorized into Host-based IDS (HIDS) and Network-based IDS (NIDS). NIDS monitor network

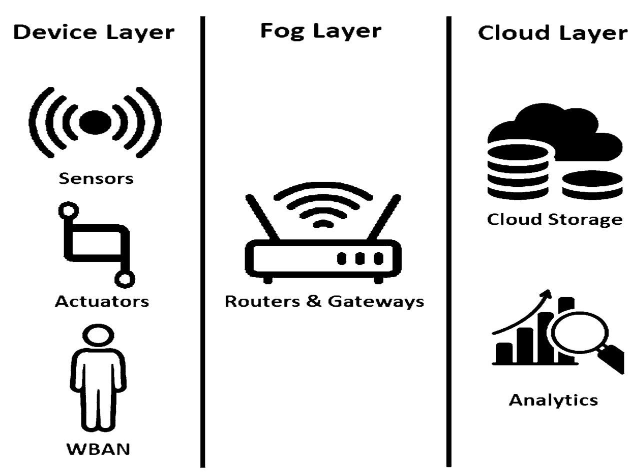


Fig. 1 3-Layered architecture of IoMT (Kulshrestha et al. 2023)

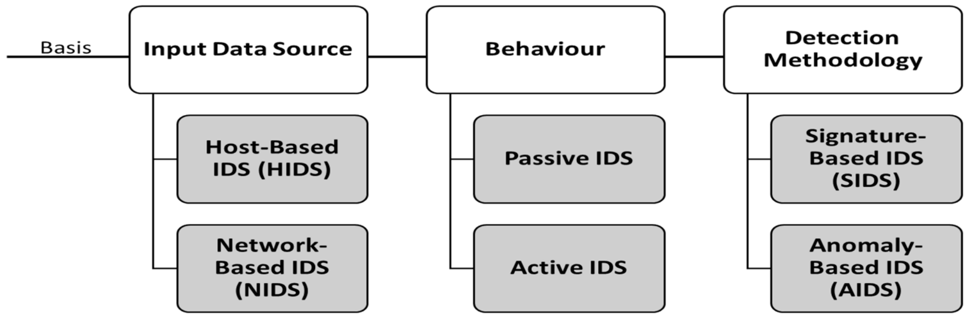


Fig. 2 Classification of IDS (Liu et al. 2018)

traffic and identify intrusions by analyzing the traffic patterns, while HIDS monitor activities on individual hosts or endpoints to detect anomalies or attacks. On the behavioral basis, it is categorized into Passive IDS and Active IDS. Passive IDS record the attack and may trigger an alert, whereas Active IDS modify the environment to thwart the threat. On the basis of detection methodology, IDS has been categorized into Signature-based IDS (SIDS) and Anomaly-based IDS (AIDS). SIDS use known patterns or signatures of attacks to detect intrusions. AIDS leverages the machine learning methods to acquire knowledge about the typical patterns of network behavior and identify deviations from the norm, which could indicate an attack.

The effectiveness of an IDS can be assessed through diverse metrics that include detection rate, false alarm rate, response time, and scalability. Detection rate measures the ability of an IDS to detect intrusions, while the false alarm rate measures the number of false positives generated by the system. Response time measures the time taken by the IDS to detect and respond to an intrusion, while scalability measures the ability of the IDS to handle large volumes of network traffic. The next subsection provides an insight to some existing ML based IDS.

2.2 Existing ML based IDS models

ML-based IDS uses supervised, unsupervised or semi-supervised learning techniques to learn from network traffic data to detect intrusions. In supervised learning, the IDS is trained on labeled data, where each data point is categorized as either normal or an attack. In unsupervised learning, the IDS learns the underlying structure of the data and identifies anomalies or outliers, which could indicate an attack. In semi-supervised learning, the IDS is trained on a combination of labeled and unlabeled data. ML-based IDS can be more effective than traditional signature-based IDS, as it can detect unknown attacks and also adapt to new attack patterns.

DL-based IDS uses neural networks to acquire intricate patterns and correlations within network traffic data. DL can be used to develop a more advanced IDS that can identify complex attacks such as zero-day attacks, polymorphic attacks and stealthy attacks. DL-based IDS can also handle high-dimensional data and identify subtle changes in network traffic patterns that may be missed by traditional IDS. However, DL-based IDS require large amounts of training data and may be computationally intensive.

ML and DL-based IDS have shown great promise in improving the effectiveness of IDS in detecting and responding to cyber-attacks. Some of the previous attempts regarding intrusion detection systems are briefly discussed in Table 1, which mentions the different approaches for a similar problem using ML & DL methods.

Both ML and DL driven IDSs encounter a range of obstacles, including the requirement for substantial labeled data, the necessity for substantial computational capabilities and the potential for false positives and false negatives. Specifically, DL-based IDS are susceptible to adversarial attacks, wherein an attacker can manipulate network traffic to bypass detection by the IDS. Ensemble learning techniques can be used to address such problems. These techniques can be used to obtain a trade-off between the bias and variance by combining models with different strengths and weaknesses. Ensemble learning can also improve the interpretability of a model by combining models having different feature selection or feature engineering strategies. In the following subsection, a concise overview is provided regarding the ensemble learning techniques employed within this paper.

2.3 Ensemble learning

An ensemble method refers to a machine learning technique that merges predictions from multiple models with the purpose to achieve superior performance compared to individual models. Ensemble learning techniques aim to achieve trade-offs between bias and variance, improve generalization performance and increase accuracy. Ensemble learning techniques such as Bagging and Boosting have been used in the proposed IoMT system. Bagging, i.e. Bootstrap Aggregation, involves training several models independently on different random subsets of training data and then use the voting/averaging mechanism to make the final prediction. Whereas, Boosting involves training several models sequentially, with each model attempting to correct mistakes of the

Table 1 Some ML and DL IDSs

References	Used approach
Khan et al. (2023)	IDS based on recurrent neural network and gated recurrent units (RNN-GRU) with two optimization techniques, Adam and Adamax
Sarkar et al. (2023)	Improved squirrel search algorithm(ISSA) is used to model the IDS with Modified-deep belief network(MDBN) on the UNSW-NB15 dataset
Kulkarni and Jaiswal (2023)	Neural Network based IDS was modelled with extended Kalman Filter as a backpropogation strategy
Alabsi et al. (2023)	To detect DDoS and DoS attacks on IoT networks, the Conditional Tabular Generative Adversarial Network(CTGAN) based IDS was modelled
Abbas et al. (2022)	Ensemble learning by using Logistic Regression (LR), NB, Decision Tree (DT) for voting classifier
Lee et al. (2021)	Multiclass classification model using Convolutional Neural Network (CNN)
Mehmood et al. (2021)	RF recursive feature elimination for optimal feature selection and SVM and Adaptive Neuro-Fuzzy System to categorize the probe
Agarwal et al. (2021)	Implemented Deep Neural Network(DNN) using whale optimization algorithm for feature selection
Swarna Priya et al. (2020)	Compared k-nearest neighbor (KNN), DNN, Naïve Bayes (NB), Random Forest (RF), SVM; feature engineering using Principal Component Analysis (PCA) and Grey Wolf
Kintzlinger et al. (2020)	Layered security scheme structure; First five layers uses statics rule base and 6th layer uses one class Support Vector Machine (SVM)
Alrashdi et al. (2019)	FBAD, i.e. Fog-based attack detection, framework was used using an Ensemble of Online Sequential Extreme Learning Machine (EOS-ELM)

previous model whereafter the final prediction is then made by combining the prediction of all models.

Overall, ensemble learning can be a powerful approach for improving the accuracy, robustness, and generalizability of machine learning models, and is particularly useful in situations where the use of a single model could result in over-fitting/under-fitting. The proposed model uses ensemble techniques on the ToN_IoT dataset and its performance is compared with the traditional machine learning models. The proposed ML based IoMT system is discussed next.

3 ML based IDS for IoMT

The methodology used by the proposed ML based IoMT system is illustrated in Fig. 3. The proposed methodology comprises of three stages: Data Pre-Processing, Training and Testing and performance evaluation. The proposed ML based IoMT system uses the ToN_IoT Network dataset, which is pre-processed by first imputing the missing values. This is followed by selecting important features using the tree based classifier for the relative importance score for each feature and then converting the categorical features into numerical features using label encoding and one-hot encoding. Subsequently, the refined data is divided into training and testing subsets in a 7:3 proportion. Following this, a fivefold cross-validation technique is implemented on the training dataset to ascertain the optimal hyperparameters. With the determined optimal hyperparameter values, the model is then trained using the complete training dataset. Finally, the trained model is subjected to testing using the designated testing dataset, and its efficacy is assessed through performance metrics such as Accuracy, Precision, Recall, F1-Score, FDR and FPR. The process is repeated for each classification model considered by the ML based IoMT system.

Next, the ToN_IoT dataset used by the proposed ML based IDS for IoMT is discussed.

3.1 ToN_IoT dataset

The proposed ML based IDS for IoMT uses the ToN_IoT Network dataset, which includes heterogeneous data collected from telemetry datasets of IoT sensors, operating system data of Windows and Ubuntu OS, and network traffic datasets. This dataset was created by Cyber Range and IoT Labs, the School of Engineering and Information technology (SEIT), UNSW Canberra at the Australian Defense Force Academy (ADFA) (Moustafa 2021). The ToN_IoT Network dataset has 46 features with mixed categorical and numerical values. It has 461,043 instances out of which 300,000 are labeled normal and 161,043 are labeled attack under the feature name *'label'*. The *types* of attacks considered in the dataset, along with their number of instances, are shown in Fig. 4. The dataset is preprocessed and the steps involved in pre-processing are briefly discussed in the next subsection.

3.2 Pre-processing

Pre-processing the ToN_IoT Network dataset involves cleaning, transforming, and preparing the data for analysis or

Fig. 3 Proposed Methodology

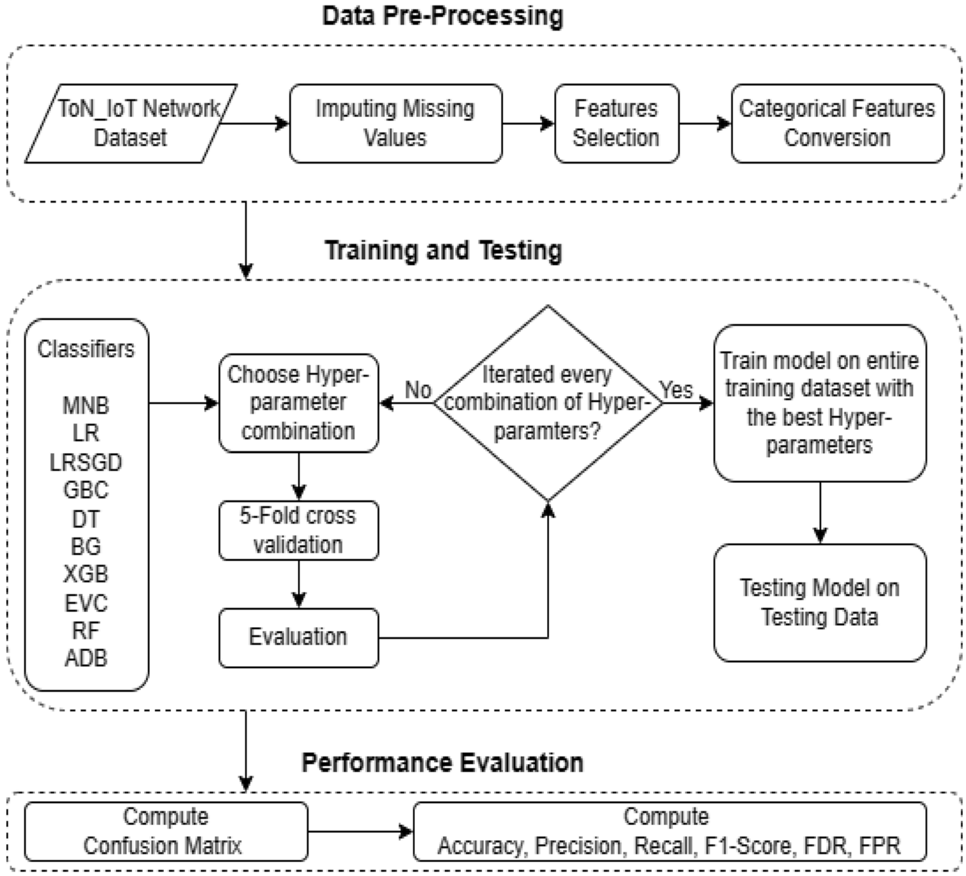
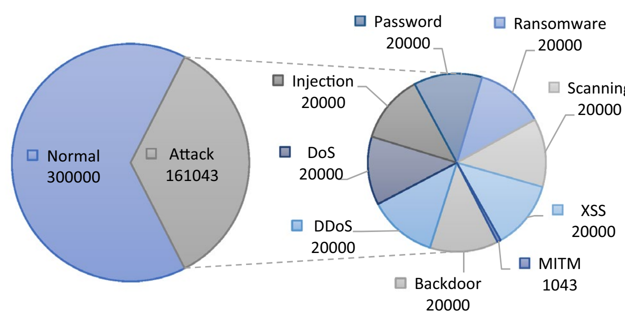


Fig. 4 No. of instances of different attack types in ToN_IoT Network dataset (Moustafa 2021)



machine learning algorithms. The pre-processing procedures for the ToN_IoT dataset are outlined below:

(a) Imputing missing values

The features 'dns_query', 'dns_AA', 'dns_RD', 'dns_RA' and 'dns_rejected' have more than 79% missing values, whereas features namely 'ssl_version', 'ssl_cipher', 'ssl_resumed', 'ssl_subject', 'ssl_issuer', 'http_trans_depth', 'ssl_established', 'http_uri', 'http_method', 'http_version', 'http_user_agent', 'weird_name', 'weird_addl', 'http_resp_mime_types', 'http_orig_mime_types' and 'weird_notice' have more than 99% missing values. So, these features are simply excluded from the dataset. One categorical feature 'service' has more than 60% of missing values but instead of dropping it, the missing values are replaced by a variable 'unknown'.

(b) Important feature selection

The features namely *'src_ip', 'src_port', 'dst_port'* and *'dst_ip'* are needed to be excluded from the features set because they can mislead the analysis. For example, if the model learns that the specific IP is malicious, it can then classify all the data points that contain that IP, as malicious. This may lead to a high false detection rate.

Out of the remaining features, some features which are more relevant, will be selected. Feature selection can be done using Univariate selection, Tree based Feature importance and Correlation matrix. In this paper, the Tree based Feature importance is used. An Extremely Randomized Trees Classifier (Extra Tree Classifier) is used to get the relative feature importance score (Pedregosa et al. 2011). Higher scores would indicate more important features.

(c) Conversion of categorical features

The approach of One-hot encoding is employed after Label encoding to convert the categorical dataset values to numerical (Geron 2019). There are three features *'service', 'proto'* and *'conn_state'* which are One-hot encoded.

3.3 Training and testing

The proposed ML based IDS for IoMT is trained using the ToN_IoT Network dataset. Classification based ML techniques such as Multinomial Naive Bayes (McCallum and Nigam 1998), Logistic Regression (Cox 1958), Logistic Regression with Stochastic Gradient Descent (Bottou 2010), Linear Support Vector Classification (Cortes and Vapnik 1995), Decision Tree (Quinlan 1986), Ensemble Voting Classifier (Littlestone and Warmuth 1994), Bagging (Breiman 1996), Random Forest (Breiman 2001), Adaptive Boosting (Freund and Schapire 1995), Gradient Boosting (Friedman 2001) and Extreme Gradient Boosting (Chen and Guestrin 2016) have been used to train the dataset. These techniques are briefly discussed below (Geron 2019):

Multinomial Naive Bayes is a probabilistic classification algorithm that is widely used in natural language processing tasks, such as text classification and sentiment analysis. It is based on the Bayes' theorem, which calculates the probability of a certain class given a set of features. In Multinomial Naive Bayes, the features are discrete and represent word counts or frequencies.

Logistic Regression is a statistical method used to analyze a dataset in which there are one or more independent variables that determine an outcome or a dependent variable that is binary or categorical in nature. It involves the utilization of a logistic function to convert a linear combination of input variables into a score that indicates the probability of an outcome occurring.

Logistic Regression with Stochastic Gradient Descent is a variant of the Logistic Regression that uses Stochastic Gradient Descent optimization to update the model's parameters. This algorithm is particularly useful for large datasets because it updates the model's parameters incrementally, rather than computing the gradient on the entire dataset in one go.

Linear Support Vector Classification is a linear classification algorithm that tries to find the hyperplane that best separates the data points of different classes. The hyperplane is defined by a set of parameters that are learned during the training process. The algorithm works by maximizing the margin between the hyperplane and the closest data points of each class.

Decision Tree is a simple and powerful classification algorithm that creates a tree-like model of decisions and their possible consequences. It works by recursively splitting the data into subsets based on the values of the input variables, until the subsets are homogeneous with respect to the target variable.

Ensemble Voting Classifier is an ensemble learning algorithm that combines the predictions of multiple models using a voting mechanism. Hard vote ensemble (also known as majority voting) involves predicting the class that receives the highest number of votes from the base models. Soft vote ensemble, on the other hand, involves computing the probabilities of each class predicted by each base model and then averaging these probabilities across all the models. The final classification reveals the class having the highest average probability.

Bagging is an ensemble learning technique that employs multiple models trained on different subsets of the train-

Table 2 List of hyper-parameters considered for each model

Models	Alias	List of hyper-parameters
Multinomial Naïve Bayes	MNB	alpha
Logistic Regression	LR	penalty, C(regularization), solver, max_iter
LR with Stochastic Gradient Descent	LRSGD	loss, penalty, alpha, learning_rate, eta0, class_weight
Linear Support Vector Classification	LSVC	penalty, loss, C(regularization)
Decision Tree	DT	min_samples_split, criterion, max_depth, min_samples_leaf
Ensemble Voting Classifier	EVC	voting
Bagging	BG	n_estimators, max_samples, max_features, bootstrap
Random Forest	RF	max_depth, min_samples_leaf, criterion, min_samples_split, n_estimators, bootstrap
Gradient Boosting Classifier	GBC	loss, min_samples_leaf, n_estimators, subsample, learning_rate, max_depth, criteria
Extreme Gradient Boosting	XGB	learning_rate, n_estimators, min_child_weight, gamma, max_depth
Adaptive Boosting	ADB	algorithm, n_estimators, learning_rate

Table 3 Evaluation criteria

Definition	Formulae	Criteria
Accuracy is the number of correct predictions over all predictions	$\frac{TP+TN}{TP+TN+FP+FN}$	Highest
Precision is the how many of positive predictions made are correct	$\frac{TP}{TP+FP}$	Highest
Recall is how many positive cases the classifier correctly predicted over all the positive cases in the data	$\frac{TP}{TP+FN}$	Highest
F1-score is the harmonic mean of recall and precision	$2\left(\frac{Precision \times Recall}{Precision+Recall}\right)$	Highest
False Detection Rate (FDR) is the ratio of false positive classifications to the total number of positive classifications	$\frac{FP}{TP+FP}$	Lowest
False Positive Rate (FPR) is calculated as the proportion of negative events that are incorrectly classified as positive, divided by the total number of true negative events	$\frac{FP}{TN+FP}$	Lowest

ing data. It randomly selects and trains models using bootstrapping (sampling with replacement). The ultimate prediction is determined by averaging the predictions generated by each individual model.

Random Forest is an ensemble learning algorithm that combines multiple decision trees to create a strong classifier. It works by randomly sampling the input variables and the training data for each tree, and averaging the predictions of all the trees. This approach reduces overfitting and thereby increases the accuracy of the model.

Bagged Decision Trees (canonical bagging) and Random Forest both create multiple decision trees using bootstrap samples of the training data. The difference lies in how they select the features to split on at each node of the tree. In bagged decision trees, all the available features are considered for splitting, while in random forest, a random subset of features is considered for splitting. This randomness reduces the correlation between the trees, making them more diverse thus reducing overfitting.

Gradient Boosting Classifier is a powerful ensemble learning algorithm that combines several weak learners (decision trees) to create a strong classifier. It works by iteratively adding decision trees that try to correct the errors of the previous trees. The final prediction is obtained by combining the predictions of all the trees.

Extreme Gradient Boosting is a variant of Gradient Boosting that uses a more regularized model to prevent overfitting. It also includes additional features such as handling missing values and parallel processing.

Adaptive Boosting is an ensemble learning algorithm that combines multiple weak learners to create a strong classifier. It works by assigning weights to the training data points and adjusting the weights based on the performance of the previous models. The final prediction is obtained by combining the predictions of all the models weighted by their performance.

The choice of hyperparameters can have a significant impact on the performance of the model, and finding the optimal values is essential to achieve best results. Hyperparameters are parameters that are set before training the model, such as the learning rate, the regularization parameter, the number of hidden layers in a neural network, and the number of trees in a random forest. In the proposed ML based IDS for IoMT, hyperparameter tuning is performed. Hyperparameter tuning is the process of selecting the optimal hyperparameters for a machine learning algorithm. Table 2 provides the list of models used, along with the corresponding sets of hyper-parameters. The names of the hyper-parameters mentioned are from python sklearn library (Pedregosa et al. 2011).

For each of the classification models, fivefold cross validation on the training dataset is performed. Hyperparameter tuning is carried out and the models are validated on different values of the respective hyperparameters, as given in Table 2. This is followed by testing the classification model considering the best hypermeter values obtained after validation. Next, based on the testing, the performance is evaluated for each of the classification models. Performance evaluation is discussed next.

3.4 Performance evaluation

Performance of the classification models considered in the proposed ML based IoMT system is evaluated and, based on the testing, a confusion matrix is computed. A Confusion Matrix is a structured representation that condenses the evaluation of a classification algorithm's effectiveness by comparing the anticipated labels with the factual labels within a dataset. The confusion matrix consists of four values: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). TPs are the cases where the model correctly predicts the positive class, FPs are the cases where the model predicts the positive class but the actual label is negative, TNs are the cases where the model correctly predicts the negative class and FNs are the cases where the model predicts the negative class but the actual label is positive (Geron 2019). Using the confusion matrix, various performance metrics are computed. The

Table 4 Set of Hyper-parametric values of each model

Models	Alias	Hyper-parameter	Values
Multinomial Naïve Bayes	MNB	'alpha'	0.001, 0.01, 0.1, 1, 10, 100
Logistic Regression	LR	'Penalty'	l1, l2, elasticnet, none
		'C'	0.001, 0.01, 0.1, 1, 10, 100
		'solver'	lbfgs, newton-cg, liblinear, sag,saga
		'max_iter'	100, 1000, 2500, 5000
LR with Stochastic Gradient Descent	LRSGD	'loss'	hinge, log, modified_huber, squared_hinge, perceptron
		'penalty'	l1, l2, elasticnet, none
		'alpha'	0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000
		'learning_rate'	'constant', 'optimal', 'invscaling', 'adaptive'
		'eta0'	1, 10, 100
		'class_weight'	(0.3,0.7), (0.4, 0.6), (0.5, 0.5), (0.6, 0.4), (0.7, 0.3)
Linear Support Vector Classification	LSVC	'penalty'	l1, l2
		'loss'	hinge, squared_hinge
		'C'	0.001, 0.01, 0.1, 1, 10, 100
Decision Tree	DT	'min_samples_split'	1, 2, 3, 4, 5, 6, 7, 8, 9, 10
		'criterion'	gini, entropy
		'max_depth'	1, 2, 4, 8, 9, 10, 15, 20, 25
		'min_samples_leaf'	3, 5, 10, 15, 20
Ensemble Voting Classifier	EVC	'voting'	Hard, soft
Bagging	BG	'n_estimators'	5, 10, 15, 20, 25, 30
		'max_samples'	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.8, 1.0
		'max_features'	0.1, 0.3, 0.7, 0.8, 0.85, 0.90, 0.92, 0.95, 1.0
		'bootstrap'	True, false
Random Forest	RF	'max_depth'	3, 5, 7, 9
		'min_samples_leaf'	1, 2, 4
		'criterion'	'gini', 'entropy', 'log_loss'
		'min_samples_split'	2, 5, 10
		'n_estimators'	100, 200, 400
		'bootstrap'	True, False
Gradient Boosting Classifier	GBC	'loss'	deviance, exponential
		'min_samples_leaf'	4, 5, 6
		'n_estimators'	5, 10, 15, 20
		'subsample'	0.6, 0.7, 0.8
		'learning_rate'	0.01, 0.05, 0.1, 1, 0.5
		'max_depth'	3, 9, 15
		'criteria'	friedman_mse, squared_error
Extreme Gradient Boosting	XGB	'learning_rate'	0.01, 0.05, 0.1, 0.5, 1
		'n_estimators'	5, 10, 15, 20, 25, 30
		'min_child_weight'	4, 6, 8, 10, 12
		'gamma'	0.1, 0.2, 0.3, 0.4, 0.5
		'max_depth'	3, 5, 6, 7, 8, 9, 10
Adaptive Boosting	ADB	'algorithm'	'SAMME', 'SAMME.R'
		'n_estimators'	10, 20, 30, 40, 50, 60, 70, 80, 90, 100
		'learning_rate'	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9

performance metric considered by the proposed ML based IoMT system is given in Table 3.

4 Experimental results

The machine used in this experiment was equipped with an Intel(R) Core (TM) i3-8130U processor clocked at

Table 5 Relative Importance score of features

Feature	Relative importance
'http_request_body_len'	0.000014
'http_response_body_len'	0.000121
'http_status_code'	0.000294
'missed_bytes'	0.001495
'duration'	0.015875
'dns_qclass'	0.019617
'dns_rcode'	0.021192
'service'	0.031855
'src_bytes'	0.034101
'dns_qtype'	0.040569
'dst_bytes'	0.040851
'dst_ip_bytes'	0.050101
'dst_pkts'	0.052534
'src_pkts'	0.075401
'src_ip_bytes'	0.143275
'proto'	0.205304
'conn_state'	0.267401

2.20 GHz, 12 GB of DDR4 RAM running at 2400 MHz and a 1 TB hard disk. Additionally, an NVIDIA GeForce MX130 Graphics card with 2 GB of memory was used for GPU-accelerated computations. The machine was running the Windows 11 operating system version 22H2 and all the experiments were conducted using Python 3.10, Jupyter notebook 6.4.6 and IPython 7.30.1. The hyper-parametric values considered for the classification techniques used in the proposed ML based IDS for IoMT are given in Table 4. The model is trained using the combination of the hyper-parameters enlisted, for each respective model, in the table.

The results of the experiment along with the corresponding inferences are discussed next.

(a) Features selected:

After dropping the less relevant features, because of major missing values and IP related features (which may mislead the analysis), the remaining features are tested for relative importance (Geron 2019). An Extremely Randomized Trees Classifier (Extra Tree Classifier), based decision trees, is used to get the relative feature importance (Pedregosa et al. 2011), as given in Table 5.

The relative feature importance can be visualized through a graph, as shown in Fig.5. The features selected are *'conn_state', 'proto', 'src_ip_bytes', 'dst_ip_bytes', 'src_pkts', 'dst_pkts', 'dst_bytes', 'src_bytes', 'service', 'dns_qtype', 'dns_qclass', 'dns_rcode', 'duration'.*

(b) Hyperparameter tuning:

The best hyper-parameter values are computed using the fivefold cross validations over each combination of hyper-parameters values for the respective classification techniques given in Table 4. The hyper-parameter values for which the classification techniques performs the best are given in Table 6.

(c) Performance comparisons:

The performance of the classifiers that have been considered for the ML based IDS for IoMT are compared on metrics such as accuracy, precision, recall, F1-score, FDR and FPR. These comparisons are given below:

Accuracy It can be observed from Fig. 6 that the ADB classifier has recorded the highest accuracy among all the classifiers. It predicted attacks with an accuracy of 99.1801 %. Consequently, ADB can be identified as the best performing classifier in terms of accuracy and if accuracy is considered as the key performance metric, ADB can be used to design the ML based IDS for IoMT.

Precision It can be observed from Fig. 7 that the ADB classifier has recorded the highest precision among all the classifiers. It predicted attacks with a precision of

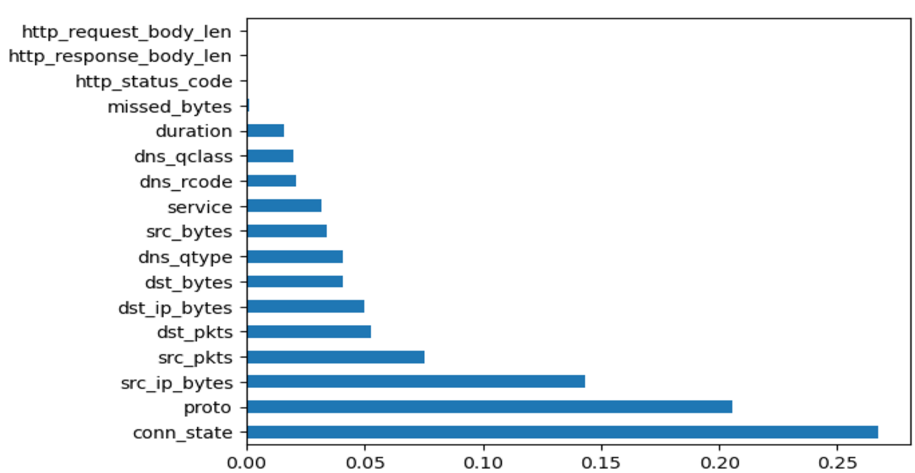


Fig. 5 Relative Importance of features

Table 6 Best hyper-parameters evaluated for each model

Model	Hyper-parameter	Best Value
MNB	'alpha'	0.001
LR	'Penalty'	none
	'C'	0.001
	'solver'	newton-cg
	'max_iter'	1000
LRSGD	'loss'	hinge
	'penalty'	elasticnet
	'alpha'	0.01
	'learning_rate'	constant
	'eta0'	100
	'class_weight'	(0.4, 0.6)
LSVC	'penalty'	l1
	'loss'	squared_hinge
	'C'	0.001
DT	'min_samples_split'	2
	'criterion'	gini
	'max_depth'	15
	'min_samples_leaf'	3
EVC	'voting'	soft
BG	'n_estimators'	30
	'max_samples'	0.8
	'max_features'	0.7
	'bootstrap'	True
RF	'max_depth'	9
	'min_samples_leaf'	10
	'criterion'	Entropy
	'min_samples_split'	2
	'n_estimators'	True
	'bootstrap'	100
GBC	'loss'	deviance
	'min_samples_leaf'	4
	'n_estimators'	20
	'subsample'	0.7
	'learning_rate'	0.5
	'max_depth'	15
	'criteria'	friedman_mse
XGB	'learning_rate'	1
	'n_estimators'	30
	'min_child_weight'	4
	'gamma'	0.2
	'max_depth'	10
ADB	'algorithm'	SAMME.R
	'n_estimators'	100
	'learning_rate'	0.7

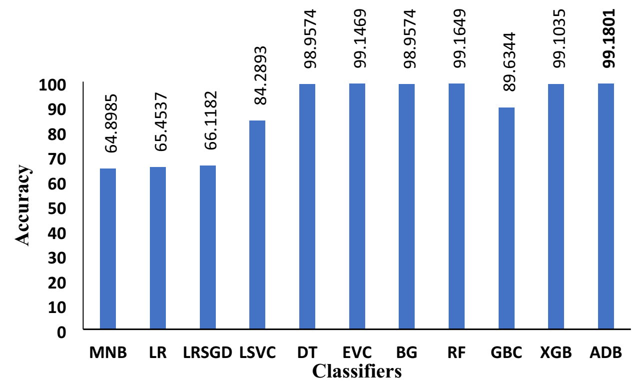


Fig. 6 Accuracy Comparison of proposed ML-based IDS for IoMT models

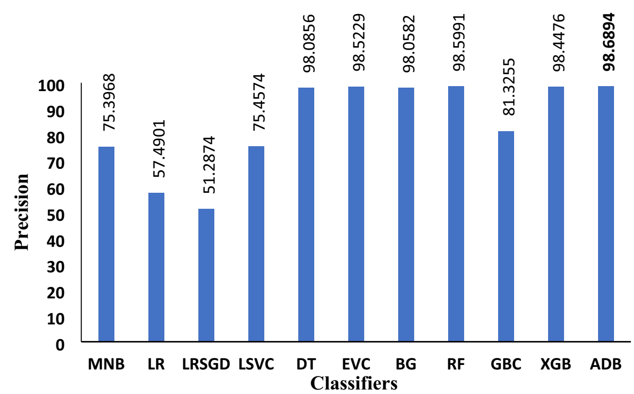


Fig. 7 Precision Comparison of proposed ML-based IDS for IoMT models

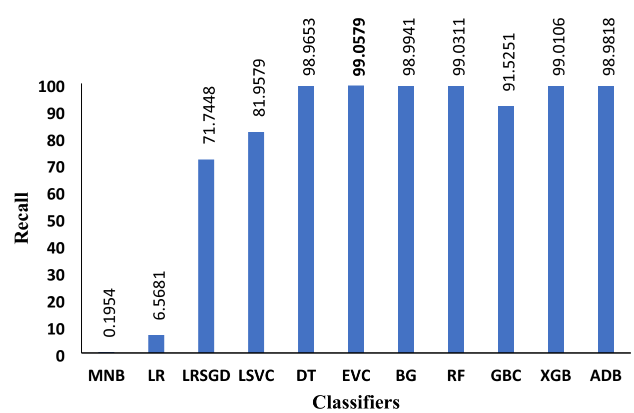


Fig. 8 Recall Comparison of proposed ML-based IDS for IoMT models

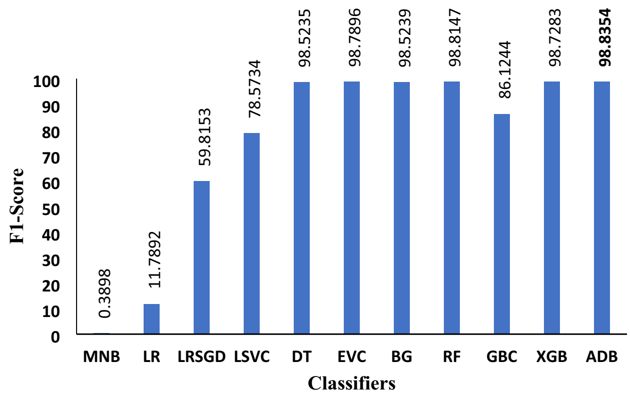


Fig. 9 F1-score Comparison of proposed ML-based IDS for IoMT models

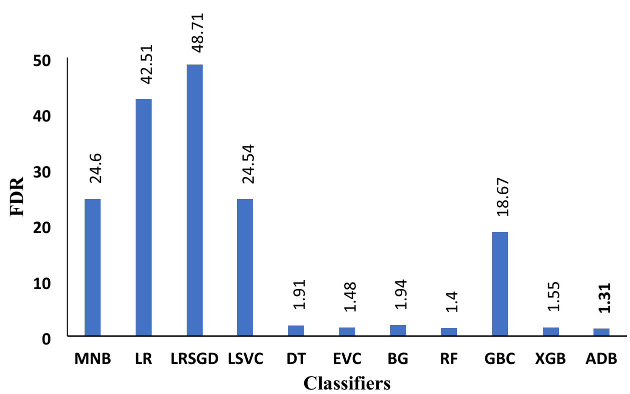


Fig. 10 FDR Comparison of proposed ML-based IDS for IoMT models

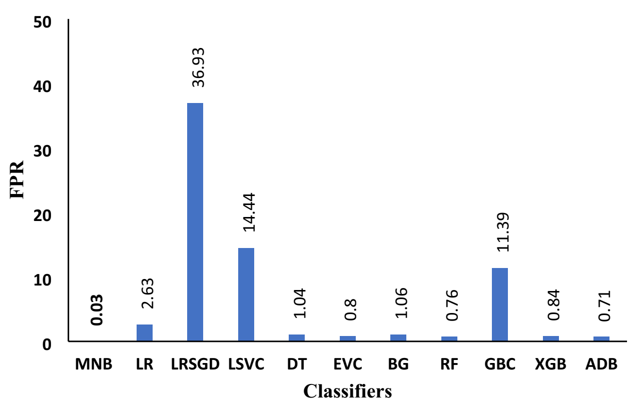


Fig. 11 FPR Comparison of proposed ML-based IDS for IoMT models

98.6894 %. Thus, it can be inferred that ADB is the best performing classifier in terms of precision and if precision is considered as the key performance metric, ADB can be used to design the ML based IDS for IoMT.

Recall It can be observed from Fig. 8 that the EVC classifier has the highest recall value of 99.0579 %, which is slightly higher than the recall value 98.9818%. of the ADB classifier. Thus, it can be inferred that EVC and ADB performed comparatively better than other classifiers in terms of recall and, thus, if recall is considered as the key performance metric, EVC or ADB can be used to design the ML based IDS for IoMT.

F1-Score It can be observed from Fig. 9 that the ADB classifier has recorded an F1-score of 98.8354%, which is comparatively higher than other classifiers. Thus, it can be inferred that ADB is the best performing classifier in terms of the F1-Score and if F1-Score is considered as the key performance metric, ADB can be used to design the ML based IDS for IoMT.

FDR It can be observed from Fig. 10 that the ADB classifier has the lowest FDR of 1.31 %, which is the least amongst all the classifiers. Consequently, ADB can be identified as the best performing classifier in terms of FDR and if FDR is considered as the key performance metric, ADB can be used to design the ML based IDS for IoMT.

FPR It is observed from Fig. 11 that the MNB classifier has the lowest FPR of 0.03% followed by the FPR value of 0.71% of the ADB classifier. Thus, it can be inferred that MNB and ADB perform comparatively better than other classifiers in terms of FPR and, thus, if FPR is considered as the key performance metric then MNB or ADB can be used to design the ML based IDS for IoMT.

Form above, it can be concluded that the ADB classifier performed the best amongst all the classifiers on all performance metrics. Therefore, ADB based IDS for IoMT is compared with the existing two ToN_IoT based models presented in (Kumar et al. 2021) and (Gad et al. 2021) respectively, which hereinafter would be referred to as Model 1 and Model 2 respectively. The Model 1 (Kumar et al. 2021) used an ensemble of DT, NB and RF while Model 2 (Gad et al. 2021) used SMOTE for feature selection. Performance metrics such as accuracy, precision, FPR and FDR, were used for comparison and are visually presented in Fig. 12. It is evident from Fig. 12 that the proposed model has a higher accuracy of 99.18% as against accuracy of 96.35% and 98.2% achieved by Model 1 and Model 2 respectively. Furthermore, the precision value exhibited by the proposed model surpasses that of the other two models. The proposed model in comparison with the other two models has the least FPR and FDR, i.e. 0.71% and 1.31% respectively. Thus it can be inferred that the proposed ML based IDS for IoMT performed comparatively better than the existing models Model

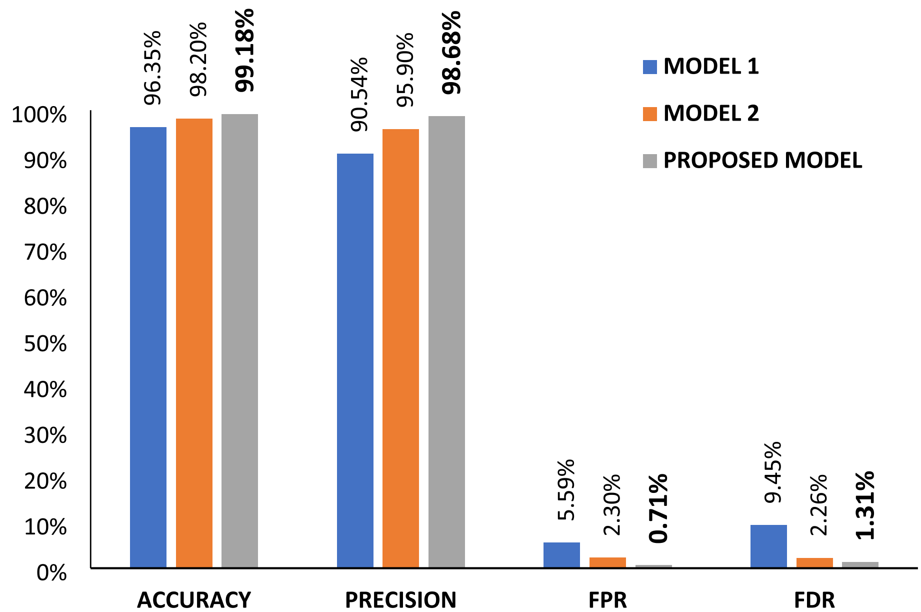


Fig. 12 Proposed ADB based IDS for IoMT model Vs. Existing ML-based models

1 and Model 2 and thus can be preferred for detecting intrusion in IoMT systems**.**

5 Conclusion

Increasing cyber threats in the healthcare domain requires a strong and reliable IDS. This paper proposes a machine learning-based intrusion detection model that was trained and tested over the cyberattack-based dataset ToN_IoT. This dataset was cleaned by excluding the columns with heavily imputed null values. The important features were selected using a tree based classifier to get a relative importance score for each feature. The categorical columns were one-hot encoded into numerical values. Hyper-parameter tuning, with fivefold cross-validation, for each of the classification models Multinomial Naive Bayes, Logistic Regression, Logistic Regression with Stochastic Gradient Descent, Linear Support Vector Classification, Decision Tree, Ensemble Voting Classifier, Bagging, Random Forest, Adaptive Boosting, Gradient Boosting and Extreme Gradient Boosting were performed to compute the best hyper-parameter values for each of these models. Thereafter, these classification models were compared, based on their best hyper-parameter values, on performance metrics such as accuracy, precision, recall, F1-score, FDR and FPR. Experimental results showed that the Adaptive boosting ensemble learning method, in comparison to other classification techniques, gave the best results on all performance metrics. Further, upon comparison with the existing two ToN_IoT based models, the Adaptive boosting based IDS for IoMT performed comparatively better on performance metrics such as accuracy, precision, FPR and FDR. Thus, it can be inferred that the Adaptive Boosting based intrusion detection model can be installed at nodes, in the architecture of IoMT, for further network traffic analysis to work as IDS.

The proposed IDS for IoMT would enhance the accessibility of high-quality healthcare, particularly for individuals living in remote locations and / or with restricted mobility. The society greatly benefits from IDS for IoMT, as it strengthens the dependability, privacy and security of medical systems and devices. By protecting patient information, ensuring the smooth functioning of healthcare operations and pre-emptively combating cyber threats, the proposed IDS for IoMT would play a critical part in advancing healthcare responses and outcomes, and enhancing patient results in our interconnected globe.

Funding None of the authors of the paper have received any funding for the work reported in this paper.

Declarations

Conflict of interest The authors declare that they have no conflict of interests.

Human and animal rights This article does not contain any studies with human participants or animals performed by any of the authors.

Informed consent Not applicable.

References

Abbas A, Khan MA, Latif S, Ajaz M, Shah AA, Ahmad J (2022) A new ensemble-based intrusion detection system for Internet of

Things. Arab J Sci Eng 47(2):1805–1819. https://doi.org/10.1007/s13369-021-06086-5

Agarwal A, Khari M, Singh R (2021) Detection of DDOS attack using deep learning model in cloud storage application. Wireless Pers Commun 127:419–439. https://doi.org/10.1007/s11277-021-08271-z

Alabsi BA, Anbar M, Rihan SD (2023) Conditional tabular generative adversarial based intrusion detection system for detecting ddos and dos attacks on the Internet of Things networks. Sensors. https://doi.org/10.3390/s23125644

Aldhaheri S, Alghazzawi D, Cheng L, Alzahrani B, Al-Barakati A (2020) DeepDCA: novel network-based detection of iot attacks using artificial immune system. Appl Sci 10(6):85. https://doi.org/10.3390/app10061909

Alrashdi I, Alqazzaz A, Alharthi R, Aloufi E, Zohdy MA, Ming H (2019) FBAD: fog-based attack detection for IoT healthcare in smart cities. In 2019 IEEE 10th annual ubiquitous computing, electronics and mobile communication conference (UEMCON), pp 0515–0522. https://doi.org/10.1109/UEMCON47517.2019.8992963

Bottou L (2010) Large-scale machine learning with Stochastic gradient descent. In Y Lechevallier, G Saporta (Eds.) Proceedings of COMPSTAT'2010, pp 177–186. Heidelberg: Physica. https://doi.org/10.1007/978-3-7908-2604-3_16

Breiman L (1996) Bagging predictors. Mach Learn 24(2):123–140. https://doi.org/10.1007/BF00058655

Breiman L (2001) Random forests. Mach Learn 45(1):5–32. https://doi.org/10.1023/A:1010933404324

Chen T, Guestrin C (2016) XGBoost: a scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 785–794. Presented at the San Francisco, California, USA. https://doi.org/10.1145/2939672.2939785

Cortes C, Vapnik V (1995) Support-vector networks. Mach Learn 20(3):273–297. https://doi.org/10.1007/BF00994018

Cox DR (1958) The regression analysis of binary sequences. J R Stat Soc Ser B 20(2):215–242

Freund Y, Schapire RE (1995). A desicion-theoretic generalization of on-line learning and an application to boosting. In P Vitányi (Ed.) Computational learning theory, pp 23–37. Berlin: Springer. https://doi.org/10.1007/3-540-59119-2_166

Friedman JH (2001) Greedy function approximation: a gradient boosting machine. Ann Stat 29(5):1189–1232. https://doi.org/10.1214/aos/101320345

Gad AR, Nashat AA, Barkat TM (2021) Intrusion detection system using machine learning for vehicular Ad Hoc networks based on ToN-IoT dataset. IEEE Access 9:142206–142217. https://doi.org/10.1109/ACCESS.2021.3120626

Geron A (2019) Hands-on machine learning with scikit-learn, keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems, 2nd edn. O'Reilly Media, Berlin

IBM Security (2022) X-Force Threat Intelligence Index 2022. IBM Corporation. Retrieved from https://www.ibm.com/downloads/cas/ADLMYLAZ

Kandasamy K, Srinivas S, Achuthan K, Rangan VP (2022) Digital healthcare-cyberattacks in asian organizations: an analysis of vulnerabilities, risks, NIST perspectives, and recommendations. IEEE Access 10:12345–12364. https://doi.org/10.1109/ACCESS.2022.3145372

Khan NW, Alshehri MS, Khan MA, Almakdi S, Moradpoor N, Alazeb A, Ullah S, Naz N, Ahmad J (2023) A hybrid deep learning-based intrusion detection system for IoT networks. Math Biosci Eng 20(8):13491–13520

Kintzlinger M, Cohen A, Nissim N, Rav-Acha M, Khalameizer V, Elovici Y, Shahar Y, Katz A (2020) CardiWall: a trusted firewall for the detection of malicious clinical programming of cardiac implantable electronic devices. IEEE Access 8:48123–48140. https://doi.org/10.1109/ACCESS.2020.2978631

Kioskli K, Fotis T, Mouratidis H (2021) The landscape of cybersecurity vulnerabilities and challenges in healthcare: security standards and paradigm shift recommendations. In Ares 21: proceedings of the 16th international conference on availability, reliability and security. Vienna, Austria, pp 136. https://doi.org/10.1145/3465481.3470033

Kulkarni DD, Jaiswal RK (2023) An intrusion detection system using extended Kalman filter and neural networks for IoT networks. J Netw Syst Manage 31(3):56. https://doi.org/10.1007/s10922-023-09748-x

Kulshrestha P, Vijay Kumar TV, Khari M (2023) Intrusion detection system for internet of medical things. In International conference on advances in IoT and security with AI (ICAISA-2023), March 24–25, 2023, New Delhi

Kumar P, Gupta GP, Tripathi R (2021) An ensemble learning and fog-cloud architecture-driven cyber-attack detection framework for IoMT networks. Comput Commun 166:110–124. https://doi.org/10.1016/j.comcom.2020.12.003

Lee JD, Cha HS, Rathore S, Park JH (2021) M-IDM: a multi-classification based intrusion detection model in healthcare IoT. Comput Mater Contin 67(2):1537–1553

Littlestone N, Warmuth MK (1994) The weighted majority algorithm. Inf Comput 108(2):212–261. https://doi.org/10.1006/inco.1994.1009

Liu M, Xue Z, Xu X, Zhong C, Chen J (2018) Host-based intrusion detection system with system calls: review and future trends. ACM Comput Surv 51(5):52

McCallum A, Nigam K (1998) A comparison of event models for naive bayes text classification. In Learning for text categorization: papers from the 1998 AAAI Workshop, pp 41–48. Retrieved from http://www.kamalnigam.com/papers/multinomial-aaaiws98.pdf

Mehmood M, Javed T, Nebhen J, Abbas S, Abid R, Bojja GR, Rizwan M (2021) A hybrid approach for network intrusion detection. Comput Mater Contin 70(1):91–107

Moustafa N (2021) A new distributed architecture for evaluating AI-based security systems at the edge: network TON_IoT datasets. Sustain Cities Soc 72:102994. https://doi.org/10.1016/j.scs.2021.102994

Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Mathieu B, Peter P, Ron W, Vincent D, Jake V, Alexandre P, David C, Matthieu B, Matthieu P, Duchesnay É (2011) Scikit-learn: machine learning in python. J Mach Learn Res 12(85):2825–2830

Quinlan JR (1986) Induction of decision trees. Mach Learn 1(1):81–106. https://doi.org/10.1007/BF00116251

Sarkar N, Keserwani PK, Govil MC (2023) A better and fast cloud intrusion detection system using improved squirrel search algorithm and modified deep belief network. Clust Comput 5:1573–7543. https://doi.org/10.1007/s10586-023-04037-3

Sharma A, Singh D (2020) Evolution of industrial revolutions: a review. Int J Innov Technol Explor Eng 9(11):2278–3075

Swarna Priya RM, Maddikunta PKR, Parimala M, Koppu S, Gadekallu TR, Chowdhary CL, Alazab M (2020) An effective feature engineering for DNN using hybrid PCA-GWO for intrusion detection in IoMT architecture. Comput Commun 160:139–149. https://doi.org/10.1016/j.comcom.2020.05.048

Zachos G, Essop I, Mantas G, Porfyrakis K, Ribeiro JC (2021) An anomaly-based intrusion detection system for internet of medical things networks. Electronics 10(21):2562. https://doi.org/10.3390/electronics10212562

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.